一、最小机器重量设计问题

1. 问题分析

给定 n 个部件，每个部件可从 m 个供应商处购买。供应商 j 提供的部件 i 有重量 w_{ij} 和价格 c_{ij}。要求在总价格不超过预算 d 的前提下，为每个部件选择一家供应商，使得所有部件的总重量最小。输出最小总重量以及每个部件所选的供应商编号（从 1 开始）。这是一个典型的 0-1 组合优化问题，解空间大小为 m^n，当 n, m 较大时无法穷举，适合使用分支限界法。

2. 解空间

- **解空间树**：一棵 n 层的多叉树。第 level 层（0 索引）对应第 level 个部件，每层有 m 个分支，分别表示选择供应商 0 ~ m-1。
- **节点状态**（代码中的 Node 结构体）：
    - level：已经处理完的部件数量（已决策部件数）。
    - cp：当前已选部件的累计价格。
    - cw：当前已选部件的累计重量。
    - path：一个 vector<int>，记录每个部件选择的供应商编号。
- **根节点**：level=0, cp=0, cw=0, path=[]。
- **叶节点**：level == n，此时得到一个完整的供应商选择方案，其总重量为 cw。

3. 剪枝函数设计

代码中采用了两类剪枝：

（1）约束剪枝（价格可行性）

扩展节点时，若 nextcp = curr.cp + c[curr.level][i] > d，则直接跳过该分支，因为已经超过预算上限。

（2）限界剪枝（重量上界）：

维护全局最优重量 bestw（初始为 INT_MAX）。

① 在从优先队列取出节点时，若 curr.cw >= bestw，则放弃该节点（因为以该节点为根的子树不可能得到更优解）。

② 扩展生成子节点时，若 nextcw = curr.cw + w[curr.level][i] >= bestw，则不入队。

（3）优先级策略：

优先队列定义为小顶堆，按 cw 升序排列。这样每次扩展的总是当前总重量最小的

节点，有助于快速找到优秀的可行解，提高剪枝效率。

4. 算法思想文字描述

采用 **优先队列式分支限界法**。

1. 初始化根节点并入队。
2. 循环取出队首节点（当前重量最小的节点）：
    - 若其重量已不小于已知最优解，则直接跳过（限界剪枝）。
    - 若 level == n（到达叶节点），则更新最优解 bestw 和 best_path。
    - 否则，对当前部件遍历所有供应商：
        - 计算新价格和新重量。
        - 若新价格不超过预算且新重量小于当前最优重量，则生成子节点并入队。
3. 队列为空时结束，输出最优重量和对应的供应商序列。

5. 算法伪代码

算法：最小重量机器设计（优先队列式分支限界）

输入：n, m, d，价格矩阵 c[n][m]，重量矩阵 w[n][m]

输出：最小总重量 bestw，每个部件的供应商编号 best_path[1..n]

```
1. 初始化小顶堆 pq（按 cw 排序）
2. 创建根节点 root = (level=0, cp=0, cw=0, path=[])
3. pq.push(root)
4. bestw = ∞
5. while pq 非空:
6.       curr = pq.pop()
7.       if curr.cw >= bestw then continue          // 限界剪枝
8.       if curr.level == n:                         // 叶节点
9.            if curr.cw < bestw:
10.                bestw = curr.cw
11.                best_path = curr.path
```

```
12.             continue
13.     for i = 0 to m-1:                           // 遍历供应商
14.             next_cp = curr.cp + c[curr.level][i]
15.             next_cw = curr.cw + w[curr.level][i]
16.             if next_cp <= d and next_cw < bestw: // 约束+限界
17.                     newNode.level = curr.level + 1
18.                     newNode.cp = next_cp
19.                     newNode.cw = next_cw
20.                     newNode.path = curr.path
21.                     newNode.path.push_back(i+1)         // 编号转为 1-based
22.                     pq.push(newNode)
23. 输出 bestw 和 best_path
```

二、查找和最小的 k 对数字

1. 问题分析

给定两个 非递减顺序 排列的整数数组 nums1 和 nums2，长度最大可达 10^5，以及整数 k（$k \le 10^4$）。要求找出和最小的 k 个数对 (u, v)，其中 u 来自 nums1，v 来自 nums2。直接枚举所有 $n \times m$ 对并排序不可行，需要利用多路归并思想，用最小堆高效求解。

2. 解空间

- **解空间**：所有可能的数对 (i, j)，其中 $i \in [0, n-1]$，$j \in [0, m-1]$。每个数对的和为 nums1[i] + nums2[j]。目标是找出和最小的 k 个。
- **节点状态**（代码中的 Element 结构体）：
  - sum：数对的和。
  - i：在 nums1 中的下标。
  - j：在 nums2 中的下标。
- **搜索方式**：将数对视为 n 条有序链，第 i 条链的元素为 (nums1[i], nums2[j])，其中 $j = 0, 1, 2, \ldots$，每条链的和随 j 递增。使用最小堆进行多路归并。

3. 剪枝函数设计

本题没有传统的剪枝函数，但利用了以下 避免重复和无效扩展 的性质：

- 初始只将每个 nums1 元素与 nums2[0] 的组合入堆（只入前 min(n, k) 条链，因为后面的链即使与最小 nums2 元素的和也太大，不可能进入前 k 小）。
- 每次弹出堆顶 (sum, i, j) 后，仅当 j+1 < m 时才将同一 i 的下一个元素 (i, j+1) 入堆。这保证了每个数对最多被生成一次，且堆始终维护当前最小的 k 个候选。

4. 算法思想文字描述

采用 **最小堆 + 多路归并** 的方法。

1. 将每个 nums1[i]（取前 min(n, k) 个）与 nums2[0] 配对，生成 (sum, i, 0) 并入堆。
2. 重复以下过程 k 次：
    - 弹出堆顶元素，将其对应的数对 (nums1[i], nums2[j]) 加入答案。
    - 如果该元素在同一链中还有下一个元素（即 j+1 < m），则将 (nums1[i] + nums2[j+1], i, j+1) 入堆。
3. 循环 k 次后，答案即为最小的 k 个数对。

由于 nums2 升序，每条链产生的和单调递增，堆顶始终是当前全局最小和，因此正确性得证。

5. 算法伪代码

算法：查找和最小的 k 对数字（多路归并+最小堆）

输入：nums1, nums2, 整数 k

输出：k 个最小数对的列表 ans（每个数对为[u, v]）

1. n = len(nums1), m = len(nums2)

2. 初始化最小堆 pq（按 sum 升序）

3. for i = 0 to min(n, k)-1:

4. pq.push((sum = nums1[i]+nums2[0], i, 0))

5. ans = []

6. while k > 0 且 pq 非空:

7. (sum, i, j) = pq.pop()

8. ans.push([nums1[i], nums2[j]])

9. if j+1 < m:

10. pq.push((nums1[i]+nums2[j+1], i, j+1))

11. k = k-1

12. 输出 ans

三、滑动谜题（2×3 滑块拼图）

1. 问题分析

给定一个 2 行 3 列的板子，上面有数字 1~5 和一个空格（用 0 表示）。每次移动可以将空格与相邻（上、下、左、右）的数字交换。目标状态为 [[1,2,3],[4,5,0]]。要求求出最少移动步数，若不可解则返回 -1。由于状态总数有限（6! = 720），使用广度优先搜索（BFS）即可获得最少步数。

2. 解空间

- **状态空间**：所有 6 个位置的全排列，共 720 种不同的板子布局。
- **节点状态**（代码中的 PuzzleState）：
  - board：一个长度为 6 的字符串，表示当前板子的展开（例如 "123450"）。
  - steps：从初始状态到达该状态所经过的移动步数。
- **邻接关系**：根据空格（0）的位置，预定义了可交换的相邻位置索引（代码中的 neighbors 数组）。例如，空格在索引 0 时可与索引 1 或 3 交换。

3. 剪枝函数设计

- **visited 集合**：使用 unordered_set<string> 记录已经访问过的状态。当扩展出一个新状态时，若该状态已在 visited 中，则不再入队，避免重复搜索和回路。
- **目标检测**：一旦当前状态的 board 等于目标字符串 "123450"，立即返回 steps（BFS 保证首次到达即为最少步数）。
- 本题不涉及更复杂的限界剪枝，因为 BFS 在完整状态空间上搜索，且状态数很少。

4. 算法思想文字描述

采用 队列式分支限界法（BFS）。

① 将初始板子转换为字符串 start，并入队，同时标记 visited。

② 循环取出队首状态：

- 若等于目标状态，返回其步数。
- 找到空格（'0'）的位置 zero_idx。
- 遍历 neighbors[zero_idx] 中所有可交换的位置 next_idx：
  - 交换字符串中的 0 和 next_idx 处的字符，得到新字符串。
  - 若新字符串未被访问过，则将其步数设为当前步数+1，并入队并标记。

③ 若队列为空仍未找到目标，返回 -1。

5. 算法伪代码

算法：滑动谜题（BFS）

输入：2×3 网格 board

输出：最少移动步数，无解返回 -1

1. 将 board 转为字符串 start（行优先拼接）

2. target = "123450"

3. 预定义邻接表 neighbors[6] = {

[0] -> {1,3},

[1] -> {0,2,4},

[2] -> {1,5},

[3] -> {0,4},

[4] -> {1,3,5},

[5] -> {2,4}

}

4. 初始化队列 q，集合 visited

5. q.push((start, 0))

6. visited.insert(start)

7. while q 非空:

8. (cur_board, steps) = q.pop()

9. if cur_board == target: return steps

```
10.     zero_idx = cur_board.find('0')
11.     for next_idx in neighbors[zero_idx]:
12.         new_board = cur_board
13.         swap(new_board[zero_idx], new_board[next_idx])
14.         if new_board not in visited:
15.             visited.insert(new_board)
16.             q.push( (new_board, steps+1) )
17. return -1
```

四、网格中的最短路径（可消除 k 个障碍）

1. 问题分析

给定一个 $m \times n$ 的网格，单元格为 0（空地）或 1（障碍）。允许最多消除 k 个障碍（即将经过的障碍视为空地）。从左上角 (0,0) 出发，每次可上、下、左、右移动一格，求到达右下角 (m-1, n-1) 的最短步数。若无法到达，返回 -1。这是一个带状态（剩余消除次数）的最短路径问题，经典解法是 BFS 扩展状态 (x, y, remain)。

2. 解空间

- **状态空间**：每个格子 (x, y) 搭配剩余的障碍消除次数 remain (0 ≤ remain ≤ k)。状态总数 ≤ $m \times n \times (k+1)$。
- **节点状态**（代码中的 GridState）：
    - x, y：当前坐标。
    - remain：还剩多少次“砸墙”能力（即可再经过多少个障碍）。
    - steps：从起点到达该状态所用的步数。
- **目标**：到达 (m-1, n-1)，且 remain 任意。

3. 剪枝函数设计

（1）**visited 数组剪枝**：

- visited[x][y] 记录到达 (x, y) 时所剩的最大 remain 值（初始化为 -1）。
- 当扩展出 (nx, ny, next_remain) 时，若 next_remain > visited[nx][ny]，则更新 visited[nx][ny] = next_remain 并允许入队；否则剪枝。
- 原理：如果之前已经用更高的剩余消除能力到达过同一格子，那么当前用更低能

力到达该格子没有优势（因为后续能通过的障碍更少），不可能得到更优解。

（2）**直接可达性剪枝**：

- 若 k >= m + n - 2，则可以直接沿右、下方向走到终点（曼哈顿距离），无需 BFS，直接返回 m+n-2。

4. 算法思想文字描述

采用 BFS（队列式分支限界） 并记录剩余消除次数。

1. 若起点即是终点，返回 0。
2. 若 k 足够大（≥ 曼哈顿距离），直接返回曼哈顿距离。
3. 初始化队列，将 (0, 0, k, 0) 入队，visited[0][0] = k。
4. BFS 循环：
    - 出队一个状态 (x, y, remain, steps)。
    - 若到达终点 (m-1, n-1)，返回 steps。
    - 枚举四个相邻格子 (nx, ny)：
        - 计算到达该格子后剩余的消除次数：next_remain = remain - grid[nx][ny]（因为若格子是障碍，需消耗一次能力）。
        - 若 next_remain >= 0 且 next_remain > visited[nx][ny]，则更新 visited 并将新状态 (nx, ny, next_remain, steps+1) 入队。
5. 若 BFS 结束未找到终点，返回 -1。

5. 算法伪代码

算法：网格中的最短路径（可消除障碍的 BFS）

输入：grid[m][n]（0 空地,1 障碍），整数 k

输出：最短步数，-1 表示不可达

1. if m==1 and n==1: return 0

2. if k >= m+n-2: return m+n-2　　// 曼哈顿距离即为最优

3. 定义队列 q，visited[m][n]全为-1

4. q.push((0,0,k,0))

5. visited[0][0] = k

6. 方向数组 dx = [-1,1,0,0], dy = [0,0,-1,1]

```
7. while q 非空:
8.     (x,y,remain,steps) = q.pop()
9.     if x==m-1 and y==n-1: return steps
10.    for i=0 to 3:
11.        nx = x+dx[i], ny = y+dy[i]
12.        if 0<=nx<m and 0<=ny<n:
13.            next_remain = remain - grid[nx][ny]    // 经过障碍则消耗 1
14.            if next_remain >= 0 and next_remain > visited[nx][ny]:
15.                visited[nx][ny] = next_remain
16.                q.push( (nx,ny,next_remain,steps+1) )
17. return -1
```

五、最佳调度问题（多机调度）

1. 问题分析

有 n 个任务，每个任务有一个处理时间 t_i。有 k 台相同的并行机器，每个任务可以分配到任意一台机器上，每台机器上的任务按顺序执行。目标是找到一种分配方案，使得 **完成全部任务的最晚时间（makespan）最小**。这是一个经典的 NP-Hard 问题，常用分支限界法求解中小规模实例。

2. 解空间

- **解空间树**：n 层的 k 叉树。第 level 层对应第 level 个任务（按降序排列），每层有 k 个分支，表示将该任务分配给第 i 台机器。
- **节点状态**（代码中的 Node 结构体）：
    - level：已分配的任务数。
    - m_times：一个 vector<int> 长度为 k，记录每台机器当前累计的任务总时间。
    - max_time：当前所有机器中的最大累计时间（即当前方案的 makespan）。
- **根节点**：level=0, m_times = [0]*k, max_time=0。
- **叶节点**：level == n，此时得到一个完整分配方案，max_time 即为该方案的完成时间。

3. 剪枝函数设计

代码中使用了以下剪枝:

（1）最优性剪枝:

维护全局最优完成时间 best_time（初始为 INT_MAX）。

① 从优先队列取出节点时，若 curr.max_time >= best_time，则跳过该节点。

② 扩展生成子节点时，若新节点的 max_time >= best_time，则不入队。

（2）任务排序:

在搜索前将任务按处理时间降序排序。这样做可以让耗时长的任务先分配，使得搜索早期就能获得较好的可行解，从而提高 best_time 的更新速度，增强剪枝效果。

（3）优先级策略:

优先队列定义为小顶堆，按 max_time 升序排列。优先扩展当前 makespan 最小的节点，有助于更快逼近最优解。

4. 算法思想文字描述

采用 优先队列式分支限界法。

1. 将任务时间数组降序排序。
2. 初始化根节点，并入队。全局最优 best_time 设为无穷大。
3. 循环取出队首节点（当前 makespan 最小的节点）:
    - 若其 makespan 不小于 best_time，则剪枝。
    - 若 level == n（所有任务已分配），则更新 best_time = min(best_time, curr.max_time)。
    - 否则，为当前任务遍历 k 台机器:
        - 将当前任务分配给机器 i，得到新节点 (level+1, 机器时间更新, 重新计算 max_time)。
        - 若新节点的 max_time < best_time，则将其入队。
4. 队列空时，输出 best_time。

5. 算法伪代码

算法：最佳调度（优先队列式分支限界）

输入：n, k, 任务时间数组 t[0..n-1]

输出：最小完成时间 best_time

1. 将 t 按降序排序

2. 初始化小顶堆 pq（按 max_time 排序）

3. 创建根节点 root = (level=0, m_times=[0]*k, max_time=0)

4. pq.push(root)

5. best_time = ∞

6. while pq 非空:

7. curr = pq.pop()

8. if curr.max_time >= best_time: continue

9. if curr.level == n: // 叶节点

10. best_time = min(best_time, curr.max_time)

11. continue

12. for i = 0 to k-1: // 尝试将当前任务分配给机器 i

13. newNode = curr

14. newNode.m_times[i] += t[curr.level]

15. newNode.level = curr.level + 1

16. newNode.max_time = max(newNode.m_times) // 计算新的最大完成时间

17. if newNode.max_time < best_time:

18. pq.push(newNode)

19. 输出 best_time